Ejercicios propuestos - Sesión 3

DropLab preparado para producción.

Junta los ejercicios 3 y 4 de la sesión 2 (los del RAG, que terminamos en la sesión 3) y los deberes de la sesión 3.
Son cuatro ejercicios, unas **tres horas** en total, y un quinto **opcional** para quien quiera ir hacia producción.

Para entender el código antes de tocarlo: abrid alumnos/sesion_3/**guia_integracion.html** (podéis revisarlo en 10 minutos pero me parece importante abrirlo 1 vez) en el navegador (va en el zip, en la carpeta de las slides).

Recorre en orden cómo se conecta todo (modelo, herramientas, flujo, API y Open WebUI), archivo a archivo y con los números de línea.

Lo de siempre: no os doy los comandos, os digo **qué quiero conseguir**. La guía de clase (guia_alumno.md), el principio de cada script y su --help son vuestro punto de partida. Si os atascáis, escribidme.

Antes de empezar: descomprimid DropLab_sesion_3.zip en una carpeta nueva y trabajad siempre desde la carpeta DropLab_sesion_3. Es autocontenido: datos, documentos (también los de junio), imágenes, los scripts de las tres sesiones, los índices de documentos ya creados (el de la sesión 2 y el ampliado con junio) y las lecturas de los PDF de junio hechas en el Jetson. Instalad las librerías con «pip install -r requirements.txt -r requirements_sesion_3.txt». Para buscar en los documentos necesitáis bge-m3 en Ollama. PaddleOCR no hace falta (requirements_ocr.txt es opcional).

Sobre los modelos: en el Jetson usamos qwen3:30b-a3b-instruct-2507-q4_K_M para el texto y qwen3-vl:8b-instruct para las imágenes. En un portátil de 16 GB: qwen3:4b-instruct y qwen3-vl:4b-instruct (las variantes
«instruct»: la otra razona antes de contestar y tarda minutos).

Ojo, se eligen de dos formas: los scripts de la sesión 2 (13, 14 y 15), con la opción --llm (mirad su --help); los de la sesión 3, con las variables de entorno de practica/sesion_3/asistente/config.py. Con 8 GB o menos: los ejercicios 1 y 2 necesitan Ollama con bge-m3 y un modelo de texto pequeño; para el 3 y el 4, los scripts 01, 05 y 08 de la sesión 3 tienen --guardada, que enseña una salida real del Jetson sin llamar a ningún modelo.

Ejercicio 1 · Montad el RAG y cambiad una decisión (**unos 50 minutos**)

En la sesión 3 (bloque I) hemos visto cómo se trocea, qué ficha lleva cada fragmento y cómo se busca son decisiones del científico de datos, no del modelo. Ahora vais a cambiar una y a medir qué pasa. Sin medir, es solo una opinión. Lo que quiero que consigáis:

- **El RAG funcionando:** el modelo de embeddings bge-m3 descargado en Ollama, el corpus preparado (con la transcripción del Jetson salen 51 fragmentos) y el índice creado. Son los scripts 10 y 11, y el orden importa.
- **Mirad cómo se trocea** un PDF y la llamada: el script 10 os enseña una fuente antes y después de cortarla. Elegid un fragmento del expediente y explicad de dónde sale cada dato de su ficha: ¿lo pone el documento o lo deduce el código?
- **La línea base.** Pasad el examen de recuperación del script 14 con k = 3 y con k = 1, y apuntad los aciertos y el MRR de las tres búsquedas. Es rápido porque no usa el LLM. (El MRR es la media de 1/posición de la
  primera fuente correcta: 1 si sale la primera, 0,5 si sale la segunda…)
- **Cambiad una decisión, solo una,** y repetid el examen. Elegid entre: (a) quitar la cabecera de contexto del texto que se indexa; (b) trocear mucho más pequeño, bajando el límite de palabras por fragmento; (c) otro modelo de embeddings, por ejemplo nomic-embed-text. Para (a) y (b) hay que tocar el script 10: buscad dónde se construye el texto que se indexa y dónde está el límite. Después, volved a crear el corpus y el índice.
- **Preguntad a los documentos** con vuestro modelo (script 13, con --llm si no es el 30B): la pregunta del Pack 4, una vuestra y una imposible. Abrid las fuentes citadas y leedlas: ¿dicen lo mismo que la respuesta?
- **Deshaced el cambio** y volved a crear el corpus y el índice antes de seguir con los demás ejercicios (y después, el script 09 de la sesión 3, que copia ese corpus en el índice ampliado).

Cosas importantes para que quede claro:

- La ficha del fragmento que elegisteis, explicada.
- Una tabla antes y después: aciertos y MRR de cada búsqueda, con k = 3 y k = 1, y qué preguntas cambian.
- Vuestra conclusión en dos o tres frases. Ojo: puede que vuestro cambio no lo empeore todo. Si mejora una cosa y empeora otra, contadlo tal cual: eso es lo interesante.
- Las tres respuestas con sus citas y, para cada una, si la fuente dice exactamente lo mismo. Si encontráis una cita real con un significado distinto, señaladla.

Ejercicio 2 · Cinco frases para dirección (**unos 20 minutos**)

Juntamos datos y documentos. En el anexo B tenéis la respuesta que escribió el analista (script 15) en la Jetson. Pasó la verificación del código: todas sus cifras salen de una consulta y todas sus citas existen.
Lo que quiero que hagáis:

- **Auditadla frase a frase:** ¿qué es un hecho, qué es testimonio y qué es hipótesis? ¿Hay alguna afirmación que no respalde ninguna fuente? ¿Qué dato importante no consultó?
- **Escribid vuestras cinco frases para dirección:** la hipótesis que priorizáis, las cifras con la consulta que las da, las fuentes (documento y página o sección; la llamada, como testimonio) y el dato que pediríais para confirmarla. Sin afirmar causas que los datos no prueban.
- **Opcional:** ejecutad el analista con vuestro modelo (script 15, --llm) y comparad su respuesta con la del Jetson.

Ejercicio 3 · El reto Malasaña, la versión que firmaríais (**unos 50 minutos**)

La pregunta del reto (reto.md): «¿Qué cambió en las ventas de Malasaña entre marzo y abril, qué aportan los documentos y la llamada, y qué demuestra este ticket?» (el ticket: data/imagenes/**ticket_abril.png**).
Con calma, quiero la respuesta que firmaríais delante de dirección. La respuesta del asistente la tenéis guardada (python practica/sesion_3/05_flujo.py --caso combinada --guardada); si podéis ejecutar los modelos, también en directo, con 05_flujo.py o con vuestra API y Open WebUI.

- **Las cifras, reproducibles:** cada una con la consulta que la da (la API, /datos/consulta, o SQL).
- **Los documentos y la llamada:** documento, página o sección, estado y fecha; la versión vigente; la llamada, como testimonio.
- **El ticket:** qué se lee, qué dice caja y el veredicto del código. Y lo que no demuestra.
- **Contradicciones, lo pendiente y una recomendación proporcionada** a la evidencia.
- **Releedla frase a frase:** ¿alguna afirma una causa, una culpa o más de lo que demuestran las evidencias?

Importante como output de la práctica:

- Una página con la plantilla (plantilla_reto.md), con su traza: qué preguntasteis y qué fuentes abristeis.
- Al lado, qué frases de la respuesta del asistente cambiasteis y por qué.

Ejercicio 4 · El cierre de junio (**unos 50 minutos**)

El segundo encargo de dirección: dos PDF de **junio** que el RAG de la sesión 2 no sabía leer (un informe con tablas que saltan de página, un gráfico y un borrador dentro, y un expediente escaneado).
En clase los leímos con PaddleOCR y los llevamos al índice con una ficha que dice «sin revisar». Ahora os toca revisar. Lo que quiero que consigáis:

- **Cuatro preguntas de junio**:
  - «¿Cuántos pares de STREET_ONE_NEGRO_41 aceptó Malasaña en junio? El borrador decía 72.» (caso junio_tabla);
  - (b) «¿Qué semana de junio tuvo el peor porcentaje de líneas completas en Malasaña? ¿Fue por la caja húmeda del albarán A-0617-04?» (junio_grafico);
  - (c) «¿Cuántos pares del albarán A-0617-04 se aceptaron al final y qué pasó con el resto?» (junio_escaneado);
  - (d) «Tras la reposición de junio, ¿faltan 18 pares en las tiendas? ¿Cuánto costó la incidencia descontando los 240 euros del transportista?» (junio_trampa).

  Para cada una, primero vuestra respuesta **leyendo los PDF** (data/documentos_junio/), con página. Después, la del asistente: python practica/sesion_3/05_flujo.py --caso <caso> --guardada (o en directo, sin
  --guardada). Leedla con las cuatro preguntas de clase: ¿es correcta?, ¿la respaldan las fuentes (abrid la cita)?, ¿dice lo que no sabe?, ¿se ve qué hizo? Para cada una: sí, a medias o no. Ojo: alguna de las cuatro tiene un error.

- **Una lectura, contra el original.** Elegid una página del expediente escaneado y comparad la lectura del OCR y la de PaddleOCR-VL (script 08 con --leer escaneado N --con ocr y --con vl, más --guardada) con la imagen del PDF. ¿Hay algún error? ¿Qué línea **del OCR** tiene la confianza más baja y por qué creéis que es?

- **Una mejora, medida.** Elegid una: (a) el caso extra_falla del script 05 («¿Cuántos pares de STREET_ONE_ARENA_41 aceptó Goya en junio…?») falla porque la pág. 3 del informe no llega a la búsqueda: añadid esa pregunta como prueba de recuperación en practica/sesion_3/examen/examen_sesion_3.json (fuente esperada: «01_DropLab_informe_reposicion_junio.pdf#3»), comprobad que falla (script 06, --parte recuperacion, no usa el LLM), proponed un arreglo (por ejemplo, repetir la cabecera de la tabla en las páginas de continuación, en practica/sesion_3/asistente/lectura.py, y rehacer el índice con el script 09) y medid antes y después; (b) el verificador del flujo da por buena una cifra si está en alguna evidencia, aunque no esté en la que se cita: proponed el cambio en practica/sesion_3/asistente/flujo.py (función verificar) y aplicadlo a las respuestas guardadas (practica/sesion_3/salidas_guardadas/flujo/): ¿cuáles marcaría y alguna correcta?; (c) otra que se os ocurra, con su forma de medirla.

Importante:

- Una tabla con las tres preguntas: vuestra respuesta y fuente, la del asistente, las cuatro preguntas (sí, a medias o no) y, si no coinciden, quién se equivocó y dónde (lectura, búsqueda, cálculo o redacción).
- La página revisada: errores encontrados (o ninguno) y la línea de confianza más baja.
- Vuestra mejora (el trozo de JSON o de código) y el resultado antes y después. Sed críticos: ¿mide de verdad lo que queréis?

Ejercicio 5 (opcional) · Hacia producción (**unos 45 minutos**)

Lo que hemos montado funciona y está medido, pero es un prototipo de aula. En el último apartado de guia_integracion.html («Para profundizar: hacia producción») tenéis lo que le faltaría para que una empresa lo usara cada día: calidad medida, verificar mejor, revisión humana, documentos al día, seguridad, registro, operación, despliegue y modelos. Lo que quiero que consigáis:

- **Elegid uno de esos temas** y escribid media página: qué problema concreto tendría DropLab hoy si lo usara así (con un ejemplo de lo que hemos visto), qué cambiaríais y en qué archivo, y **cómo sabríais que ha funcionado**.
- **Si os animáis, hacedlo en pequeño.** Por ejemplo: guardar cada pregunta, su respuesta, sus fuentes y sus tiempos en un archivo JSONL desde la operación /analizar de asistente/api.py; exigir una clave en una cabecera para usar la API (FastAPI, seguridad); o pasar 06_evaluar.py antes y después de un cambio. Con --guardada o con un modelo pequeño basta.

Si queréis más: el ticket con vuestro modelo (01_ver_imagen.py), un cliente propio de la API (04_cliente_api.py y /docs), la evaluación por piezas (06_evaluar.py) y las dos configuraciones (07_comparar.py). No cuentan para nota.

Junto a este documento tenéis una lista de enlaces útiles (ordenados por ejercicio)